



Tecnicatura Universitaria
en Programación

PROGRAMACIÓN IV

FRONT END

Unidad N° 1:
Arquitectura y Despliegue

Anexo
2° Año – 4° Cuatrimestre



Índice

Unidad 1: Arquitectura y Despliegue	3
Arquitectura cliente-servidor y el problema de escalar	3
Nginx como servidor web	4
Servidor web	4
Nginx.....	5
Proxy inverso (reverse proxy).....	6
Proxy directo (forward proxy).....	6
Definición de proxy inverso	7
Nginx como reverse proxy	7
Nginx en arquitecturas de microservicios	8
Arquitectura de microservicios	9
Nginx como gateway	9
Backend for Frontend (BFF).....	11
Load balancing con Nginx	12
Implementación práctica: Nginx como servidor web y reverse proxy con Docker	15
Estructura del proyecto.....	15
El contenedor de frontend: build en dos etapas	15
Configuración de Nginx: servidor web y reverse proxy en el mismo archivo	16
Orquestación con Docker Compose.....	17
Configuración dinámica al arrancar el contenedor	19
Qué es el despliegue y por qué es una etapa crítica	20
El despliegue en el ciclo de vida del software	21
Estrategias de despliegue: Blue-Green y Canary	21
Blue-Green Deployment	21
Canary Deployment	22
Estrategias de despliegue: Rolling Update, A/B Testing, Shadow Deployment y Feature Flags	23
Rolling Update	23
A/B Testing Deployment	25
Shadow Deployment.....	26
Feature Flags.....	26
Herramientas para el despliegue	27
Infraestructura e inmutabilidad	27
Pipelines de integración y despliegue continuo	27
Secretos y configuración	29
Monitoreo y rollback automático	29
En resumen	30

Unidad 1: Arquitectura y Despliegue

Cuando una aplicación web deja de ser un proyecto que corre en la computadora de quien la desarrolla y pasa a atender usuarios reales, aparecen preguntas que el código por sí solo no resuelve. ¿Qué recibe las peticiones cuando llegan miles al mismo tiempo? ¿Cómo se sirve el frontend ya compilado sin exponer directamente el servidor que corre la lógica de negocio? ¿Cómo se reparte el tráfico entre varias instancias de un mismo servicio? Y cuando hay una versión nueva lista para publicarse, ¿cómo se pasa de la versión vieja a la nueva sin que quien usa el sistema note una interrupción?

Esta unidad responde esas preguntas desde dos ángulos que se complementan. El primero es de **infraestructura**: qué componente se ubica entre el cliente y los servidores de aplicación, y qué roles cumple ese componente a medida que el sistema crece de un sitio simple a una arquitectura de microservicios. El protagonista de ese bloque es **Nginx**, uno de los servidores más usados en la industria para resolver exactamente ese problema. El segundo ángulo es de **proceso**: una vez que el sistema está armado, ¿de qué manera se publica una versión nueva sin arriesgar la disponibilidad del servicio? Ahí entran las **estrategias de despliegue**, los patrones que usan los equipos profesionales para liberar cambios con el menor riesgo posible.

A lo largo del primer bloque seguís el caso de la Pizzería Don Nginx, un negocio ficticio y concreto que arranca mostrando su menú en una página estática y va creciendo: suma pedidos en línea, después un backend propio, más tarde varios microservicios independientes y finalmente balanceo de carga entre instancias. Cada tema del bloque aparece porque el crecimiento de la pizzería lo necesita.

Arquitectura cliente-servidor y el problema de escalar

En su forma más simple, la web funciona con dos actores: un cliente (el navegador de quien usa el sitio) y un servidor de aplicación, que recibe la petición, ejecuta la lógica de negocio correspondiente y devuelve una respuesta.

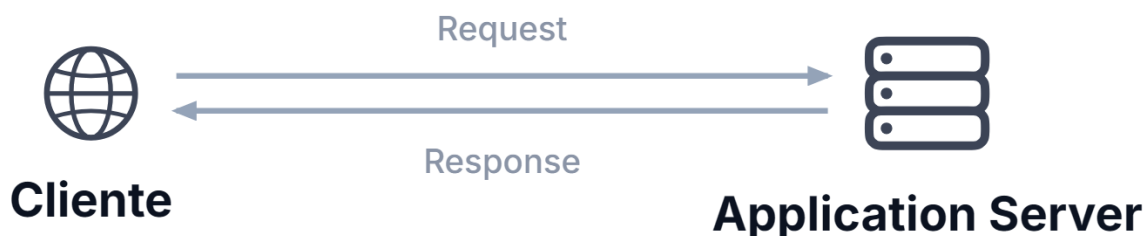


Figura 1. Elaboración propia. Flujo de solicitud y respuesta entre cliente y servidor de aplicaciones.

Mientras el sitio tiene pocos visitantes, este esquema alcanza y sobra. El problema aparece cuando el tráfico crece: el servidor de aplicación empieza a recibir muchas solicitudes al mismo tiempo, y atenderlas todas empieza a demorar. Además, ese mismo

servidor suele encargarse de tareas que no tienen nada que ver con la lógica de negocio, como entregar imágenes, hojas de estilo o archivos JavaScript, y cada una de esas tareas le quita capacidad de cómputo a lo que realmente importa: procesar pedidos, calcular precios, consultar la base de datos.

Sin un componente que organice ese tráfico, resulta difícil repartir la carga entre varios servidores, guardar en caché el contenido que no cambia seguido, o filtrar conexiones antes de que lleguen al backend. Queda abierta la pregunta de qué componente puede pararse en el medio para resolver todo eso.

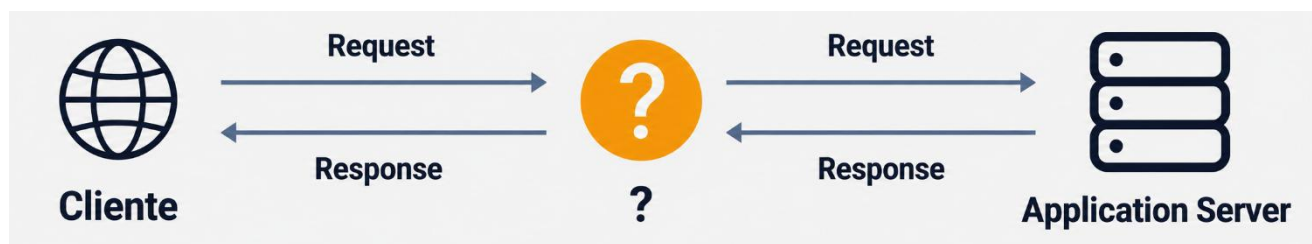


Figura 2. Elaboración propia. Intermediación de las solicitudes y respuestas entre el cliente y el servidor de aplicaciones

Esa pregunta tiene nombre, y es el hilo del resto de este bloque: un servidor **liviano** y **especializado**, capaz de manejar una enorme cantidad de conexiones simultáneas, que se ubica entre el cliente y el servidor de aplicación. **Nginx** es, hoy, una de las respuestas más usadas en la industria para ese lugar vacío.

Nginx como servidor web

Antes de ocupar ese lugar intermedio, conviene entender el rol más simple que puede cumplir un servidor: el de servidor web.

Servidor web

Un servidor web es un software que recibe solicitudes de clientes (generalmente navegadores) y responde con contenido que ya existe: páginas HTML, hojas de estilo, imágenes, scripts o videos. Cuando alguien escribe una dirección en el navegador, este envía una solicitud HTTP a ese servidor, que localiza el archivo pedido y lo devuelve tal cual está guardado.

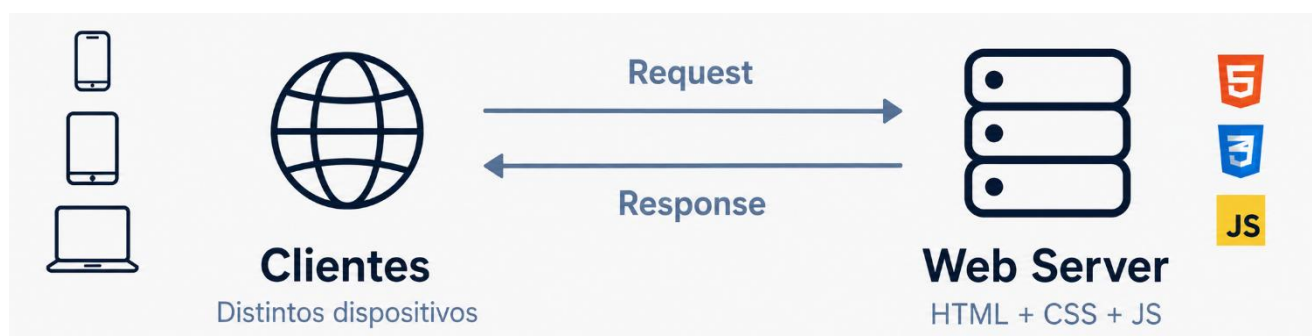


Figura 3. Elaboración propia. Comunicación entre clientes de distintos dispositivos y un servidor web.

El servidor web no genera contenido dinámico ni ejecuta lógica de negocio: se limita a entregar lo que ya existe. Esa limitación es, en realidad, lo que lo hace rápido, porque no tiene que calcular nada, solo buscar un archivo y devolverlo.

Nginx

Nginx es un software de código abierto y alto rendimiento que, además de servidor web, puede desempeñarse como proxy inverso, balanceador de carga, servidor de caché de contenido o proxy TCP/UDP. Esta unidad desarrolla en profundidad los tres primeros roles (servidor web, proxy inverso y balanceo de carga), que son los más frecuentes al desplegar una aplicación web.



Figura 4. Elaboración propia. NGINX como servidor web y proxy inverso.

Cumple el rol de servidor web con una arquitectura basada en eventos, en vez de crear un proceso o un hilo por cada conexión que recibe. Esa diferencia importa: un servidor que abre un proceso nuevo por cada cliente conectado consume cada vez más memoria a medida que crecen las conexiones simultáneas; uno basado en eventos atiende miles de conexiones con un consumo de recursos mucho más bajo. Por eso Nginx se volvió una pieza habitual en la infraestructura web moderna: es eficiente, rápido, simple de configurar y estable en producción.



Figura 5. Elaboración propia. Comunicación entre el cliente y un servidor web Nginx.

Se usa con frecuencia para servir aplicaciones modernas hechas con **React** o **Angular**: una vez compiladas, se convierten en un conjunto de archivos estáticos que Nginx entrega de forma eficiente. Por eso es común encontrar Nginx dentro de la imagen Docker de una aplicación Angular: el build genera la carpeta `dist/`, y Nginx se encarga de servir esos archivos.

```
server {
    listen 80;
    root /usr/share/nginx/html;
    index index.html;
}
```


Con esas tres líneas, Nginx escucha el puerto 80 y responde con los archivos que encuentre en `/usr/share/nginx/html`: exactamente lo que necesita una aplicación Angular ya compilada.



Para seguir el crecimiento de un caso concreto a lo largo de este bloque, pensá en la **Pizzería Don Nginx**, un negocio que decide tener presencia en la web.

En esta primera etapa, la pizzería solo quiere mostrar su menú, sus horarios y sus datos de contacto. Los archivos que arman ese sitio (HTML, CSS, imágenes) están guardados en el servidor, y Nginx los entrega directamente cada vez que alguien entra al sitio. No hay backend ni base de datos: el navegador pide una página y Nginx la sirve tal cual está guardada.



Proxy inverso (reverse proxy)

El menú en línea le funcionó bien a la pizzería, pero el negocio quiere dar el siguiente paso: que los clientes hagan pedidos desde la web. Eso ya no es contenido estático: pedir una pizza implica ejecutar lógica de negocio, guardar el pedido en una base de datos y devolver una confirmación. Nginx, tal como se usó hasta acá, no hace nada de eso, solo entrega archivos.

La solución no es reemplazar a Nginx, sino sumarle un backend (un servidor de aplicación) que se encargue de esa lógica, y hacer que Nginx decida a quién le corresponde cada solicitud: si pide un archivo estático, lo resuelve él mismo; si pide algo relacionado con pedidos, se lo pasa al backend. Ese rol tiene nombre: **proxy inverso**.

Proxy directo (forward proxy)

Antes de definir el proxy inverso conviene distinguirlo de su contraparte, porque suelen confundirse. Un proxy directo se ubica del lado del cliente: uno o varios clientes lo usan como intermediario para salir a internet. El proxy recibe la solicitud del cliente, la reenvía al servidor de destino en su nombre, y devuelve la respuesta; para ese servidor, la solicitud parece venir del proxy, no del cliente real.

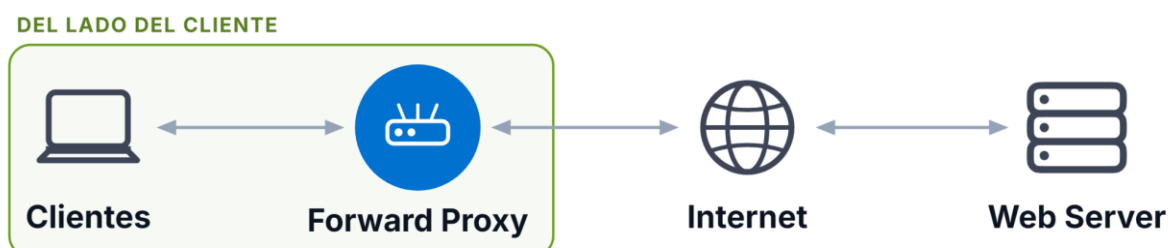


Figura 6. Elaboración propia. Funcionamiento de un Forward Proxy entre los clientes e Internet.

Dos casos concretos donde aparece este esquema:

1. Una empresa que hace pasar todo el tráfico saliente de sus empleados por un proxy corporativo, para bloquear sitios no autorizados y registrar qué se accede desde la red interna.
2. Un servicio de VPN, que oculta la dirección IP real de quien lo usa frente a los sitios que visita.

En los dos casos el proxy protege o controla al cliente, no al servidor.

Definición de proxy inverso

Un proxy inverso hace lo simétrico: se ubica del lado del servidor, y oculta al servidor real frente a los clientes. Quien hace una solicitud nunca sabe, ni necesita saber, cuántos servidores hay detrás del proxy inverso ni cómo están organizados: solo ve una única dirección pública.

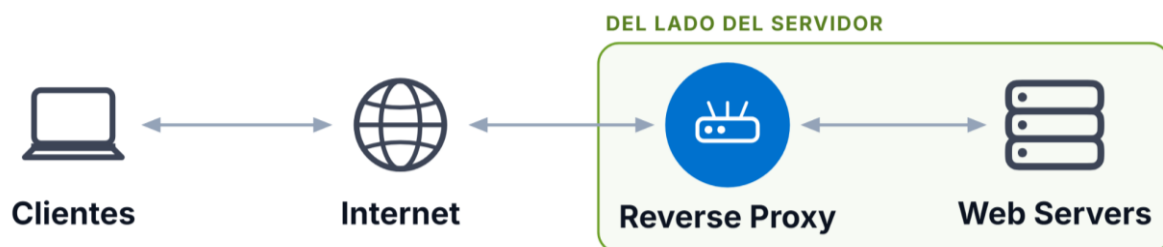


Figura 7. Elaboración propia. Funcionamiento de un Reverse Proxy entre Internet y los servidores web.

Nginx como reverse proxy

Cuando Nginx actúa como proxy inverso, se convierte en el único punto de entrada para las solicitudes de los clientes. Decide, según la ruta solicitada, si la atiende directamente (un recurso estático) o si la reenvía a un servidor de aplicación (una ruta de API).

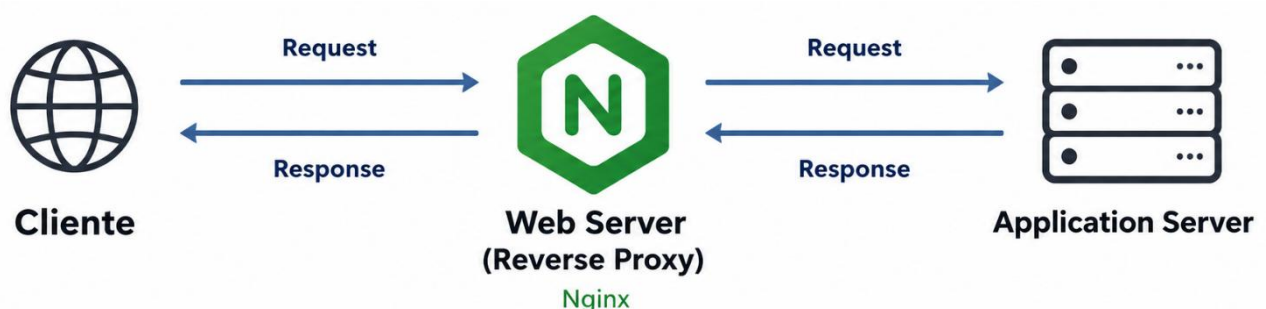


Figura 8. Elaboración propia. Funcionamiento de Nginx como Reverse Proxy entre el cliente y el servidor de aplicaciones.

Pensalo como el recepcionista de un edificio de oficinas. Cuando alguien llega preguntando por el departamento de contabilidad, el recepcionista no lo deja entrar directo a las oficinas: escucha el pedido, identifica a quién corresponde, lo comunica puertas adentro y devuelve la respuesta. Quien visita nunca tiene contacto directo con las oficinas internas, y el recepcionista controla el flujo de gente que entra al edificio. Ese es el trabajo de un reverse proxy: **intermediar, ocultar la infraestructura interna y mantener ordenado el tráfico.**

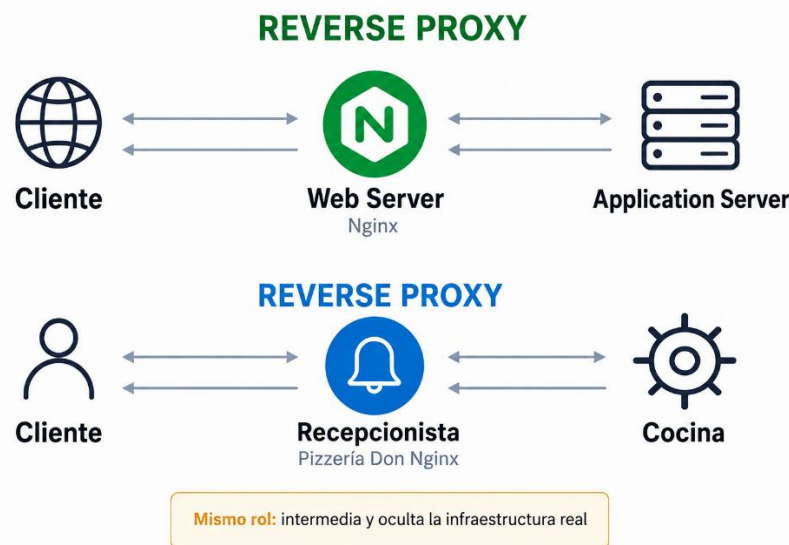


Figura 9. Elaboración propia. Analogía del funcionamiento de un Reverse Proxy mediante Nginx.

Además de intermediar, un reverse proxy suma ventajas concretas: **oculta las direcciones reales de los servidores internos**, **centraliza** en un mismo punto **el control de tráfico**, la **autenticación** y los **logs**, y permite aplicar **caché** o límites de tasa (**rate limiting**) antes de que la solicitud llegue al backend.

Volviendo a la pizzería: ahora hay un backend hecho en Spring Boot que registra pedidos y consulta la base de datos. Nginx sigue sirviendo los archivos del frontend Angular, pero cuando la solicitud es `POST /api/pedidos`, la reenvía al backend.

```
location /api/pedidos/ {
    proxy_pass http://pizzeria-backend:8080/;
}
```

El bloque `location` intercepta las solicitudes que empiezan con `/api/pedidos/`, y `proxy_pass` indica a qué servidor reenviarlas. La sección de implementación práctica, más adelante, retoma esta misma configuración con el detalle completo.

Con este cambio, cuando alguien entra a la pizzería, Nginx le entrega el frontend Angular; cuando hace un pedido, Nginx reenvía esa solicitud puntual al backend, que la procesa y guarda en la base de datos.

Nginx en arquitecturas de microservicios

El esquema anterior funciona bien con un solo backend, pero la pizzería sigue creciendo: ahora quiere separar la gestión de pedidos, el procesamiento de pagos y el registro de clientes frecuentes en servicios independientes, cada uno con su propio ciclo de desarrollo y su propia base de datos. Mantener todo eso junto en un único backend (un monolito) empieza a traer problemas de mantenibilidad: un cambio en pagos obliga a probar y volver a desplegar todo el sistema, aunque pedidos no haya cambiado en absoluto.



Figura 10. Elaboración propia. Representación de una arquitectura de MS aplicada a pedidos, pagos y gestión de usuarios/clientes.

Arquitectura de microservicios

La solución habitual es dividir la aplicación en microservicios: módulos independientes, cada uno responsable de una función puntual (pedidos, pagos, usuarios), que pueden desarrollarse, desplegarse y escalar por separado. Esa independencia resuelve el problema de mantenibilidad, pero abre uno nuevo: ¿cómo hace un cliente externo para comunicarse con varios servicios distintos sin conocer la dirección interna de cada uno?

Nginx como gateway

Ahí Nginx vuelve a cumplir el rol de proxy inverso, ahora frente a varios servicios en lugar de uno solo. Se ubica como puerta de entrada única: recibe todas las solicitudes externas y las reenvía al microservicio que corresponda según la ruta.

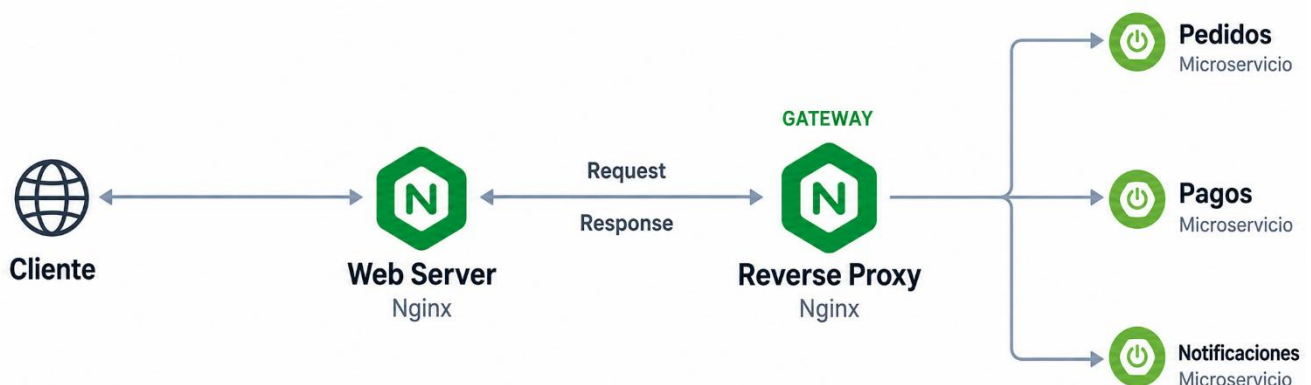


Figura 11. Elaboración propia. Arquitectura de comunicación entre cliente, servidor web, reverse proxy y microservicios.

Por ejemplo, una solicitud a `/api/pagos` se redirige al microservicio de pagos, mientras que una solicitud a `/api/usuarios` se redirige al microservicio de usuarios. Quien hace la solicitud solo conoce una dirección pública (la de Nginx); nunca necesita saber que detrás hay varios servicios distintos corriendo en contenedores separados.

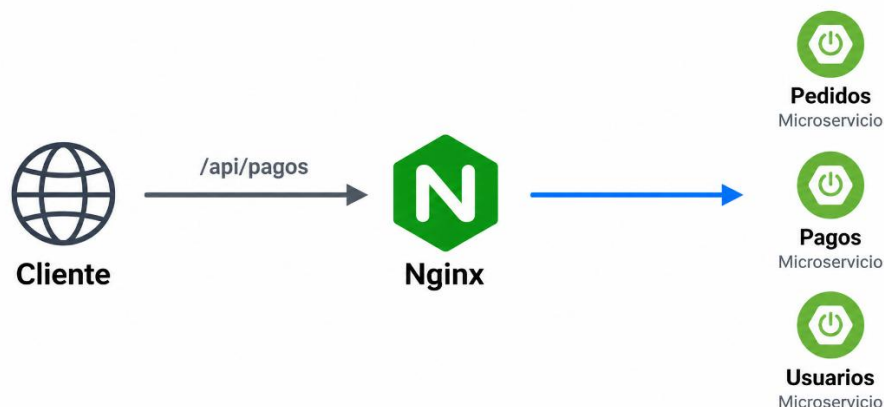


Figura 12. Elaboración propia. Enrutamiento de una solicitud /api/pagos mediante Nginx hacia el microservicio de Pagos.

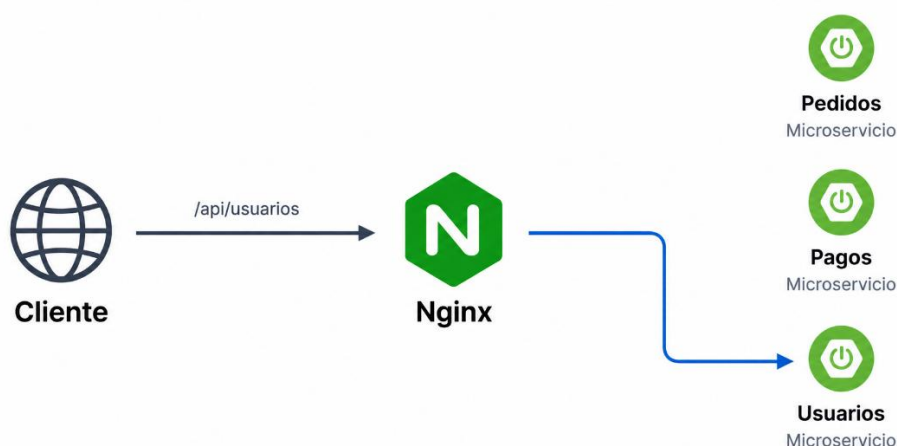


Figura 13. Elaboración propia. Enrutamiento de una solicitud /api/usuarios mediante Nginx hacia el microservicio de Usuarios.

```
location /api/pedidos/ {  
    proxy_pass http://pedidos-service:8081;  
}  
  
location /api/pagos/ {  
    proxy_pass http://pagos-service:8082;  
}  
  
location /api/usuarios/ {  
    proxy_pass http://usuarios-service:8083;  
}
```

Cada bloque `location` es una regla de ruteo independiente: la ruta determina el destino, y agregar un microservicio nuevo es tan simple como sumar un bloque más, sin tocar los que ya funcionan.

Es habitual, además, encontrar más de un Nginx en la misma infraestructura: uno dedicado a servir el frontend Angular, y otro (o varios, en capas) dedicado a rutear entre microservicios. Cada equipo, de frontend o de backend, puede así desplegar su parte de forma independiente, sin compartir el mismo punto de configuración.

Backend for Frontend (BFF)

El gateway anterior resuelve el ruteo (a qué microservicio va cada solicitud), pero no resuelve otro problema que aparece cuando una sola pantalla necesita datos de varios microservicios a la vez. Si la pantalla de inicio de la pizzería muestra en una sola vista el historial de pedidos y los pagos pendientes, Angular tendría que hacer dos llamadas separadas (una a pedidos, otra a pagos) y combinar las respuestas del lado del cliente.

Un patrón habitual para resolver esto es el **Backend for Frontend (BFF)**: un servidor dedicado a una experiencia de frontend puntual, que se ubica entre el gateway y los microservicios. El BFF recibe una única solicitud, llama a los microservicios que necesite, combina y filtra los datos, y devuelve una respuesta ya armada para esa pantalla específica. A diferencia del gateway, que solo redirige sin tocar el contenido, el BFF sí conoce la forma en que el frontend necesita los datos.

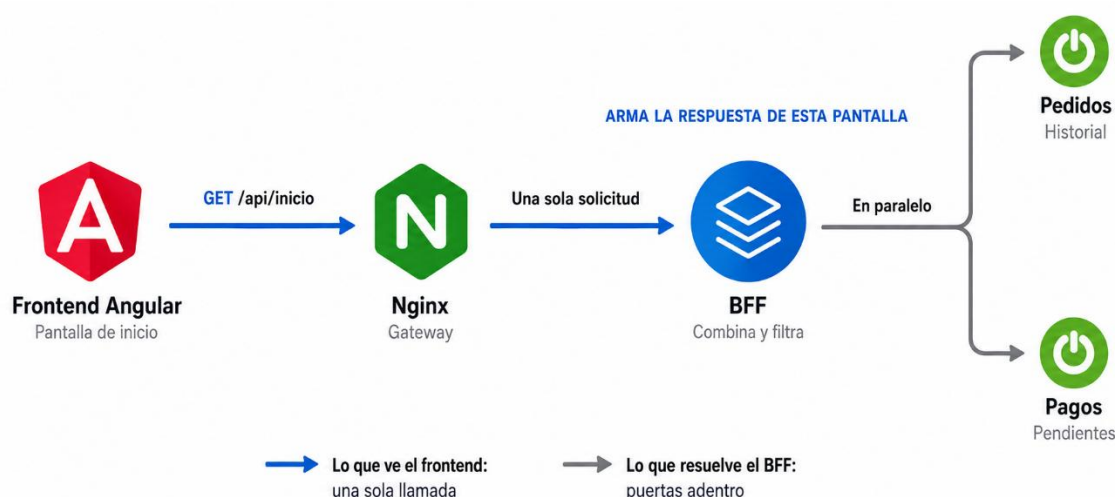


Figura 14. Elaboración propia. Uso del patrón BFF para consolidar en una única respuesta la información de múltiples microservicios.

La asimetría del diagrama es el punto: a la izquierda del BFF hay una sola flecha, a la derecha hay tantas como microservicios haga falta consultar. Esa diferencia es la que el frontend deja de tener que resolver.

```
app.get('/api/inicio', async (req, res) => {
  const [pedidos, pagos] = await Promise.all([
    fetch('http://pedidos-service:8081/historial'),
    fetch('http://pagos-service:8082/pendientes'),
  ]);

  res.json({ pedidos: await pedidos.json(), pagos: await pagos.json() });
});
```

Con este BFF, la pantalla de inicio de la pizzería hace una sola llamada a `/api/inicio`, y es el BFF quien combina el historial de pedidos con los pagos pendientes antes de responder. El frontend no necesita saber que esos datos vienen de dos microservicios distintos, ni encargarse de combinarlos él mismo.

Load balancing con Nginx

El gateway resuelve a qué microservicio va cada solicitud, pero no resuelve qué pasa cuando un mismo microservicio no da abasto. Durante los fines de semana, el microservicio de pedidos de la pizzería recibe muchas más solicitudes que durante la semana, y una sola instancia empieza a responder cada vez más lento.

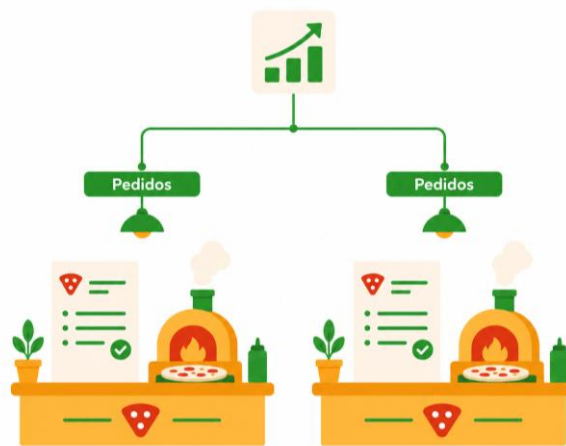


Figura 15. Elaboración propia. Escalabilidad horizontal del microservicio de Pedidos mediante múltiples instancias.

La solución no es agrandar esa instancia indefinidamente, sino sumar más instancias del mismo microservicio y repartir el tráfico entre todas: eso es el balanceo de carga (load balancing). El objetivo es que ningún servidor quede sobrecargado mientras otros están ociosos, y que si una instancia falla, el tráfico se redirija automáticamente a las que siguen respondiendo.

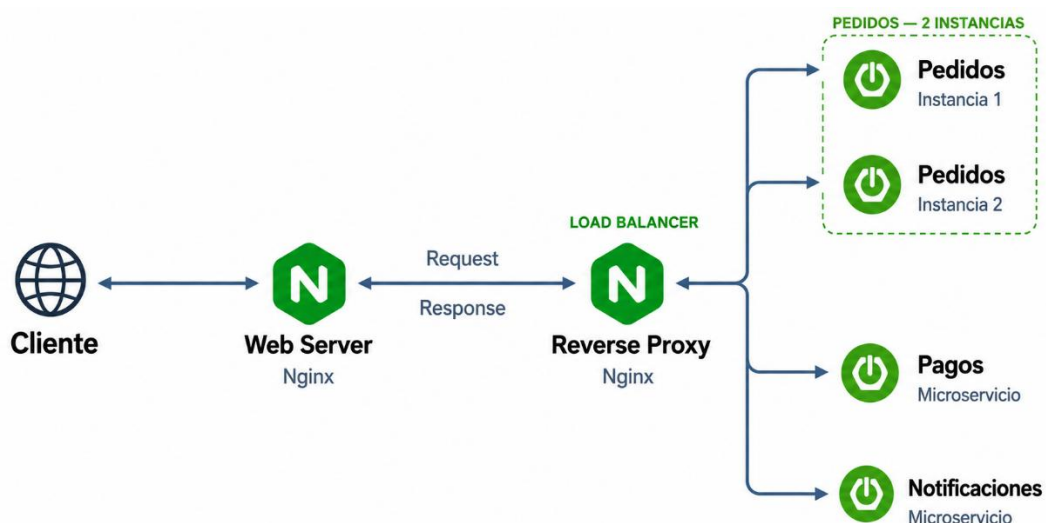


Figura 16. Elaboración propia. Balanceo de carga y escalabilidad horizontal del microservicio de Pedidos mediante múltiples instancias.

Nginx puede cumplir este rol de forma nativa, sin herramientas adicionales, gracias a la misma arquitectura basada en eventos que ya lo hace eficiente como servidor web y como proxy inverso. Para usarlo, se agrupan varias instancias bajo un bloque `upstream`:

```
upstream pedidos_backend {
    least_conn;
    server pedidos-service-1:8081;
    server pedidos-service-2:8081;
}

location /api/pedidos/ {
    proxy_pass http://pedidos_backend/;
}
```

`upstream` define el grupo de servidores disponibles, y `proxy_pass` ahora apunta al grupo en lugar de a un único servicio. La directiva `least_conn` (una de varias posibles) le indica a Nginx qué algoritmo usar para elegir, en cada solicitud, a qué instancia del grupo enviarla.

Algoritmo	Directiva	Cómo elige el servidor	Cuándo conviene
Round Robin (por defecto)	Sin directiva adicional	Reparte las solicitudes en orden circular, una a cada servidor por turno	Instancias con capacidad similar y solicitudes parejas entre sí
Weighted Round Robin	<code>server host weight=3;</code>	Igual que Round Robin, pero las instancias con mayor <code>weight</code> reciben una proporción mayor de solicitudes	Instancias con distinta capacidad de cómputo entre sí
Least Connections	<code>least_conn;</code>	Envía la solicitud a la instancia con menos conexiones activas en ese momento	Carga despareja, o solicitudes que tardan tiempos muy distintos entre sí
IP Hash	<code>ip_hash;</code>	Asigna cada cliente siempre a la misma instancia, según su dirección IP	Hace falta mantener sesiones o estado temporal asociado a un cliente (sticky sessions)

Tabla 1. Elaboración propia. Algoritmos de balanceo de carga disponibles en Nginx y criterios de uso.

Gráficamente:

Round Robin (por defecto):

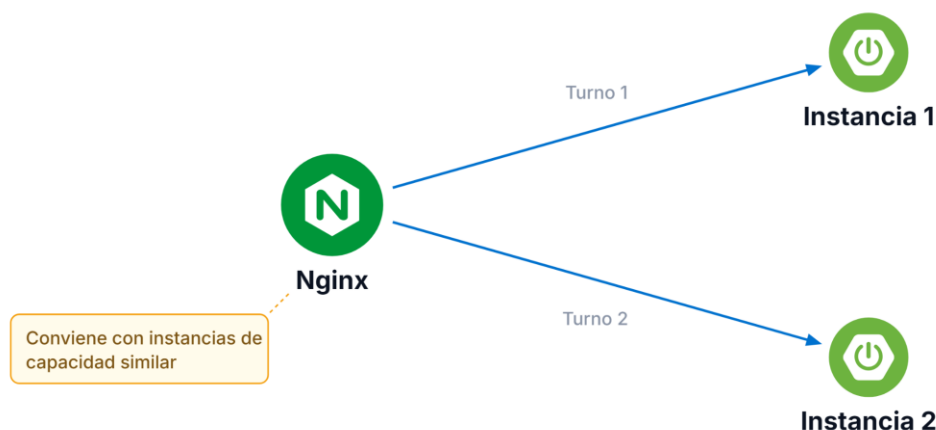


Figura 17. Elaboración propia. Distribución de solicitudes mediante el algoritmo Round Robin entre múltiples instancias.

Weighted Round Robin:

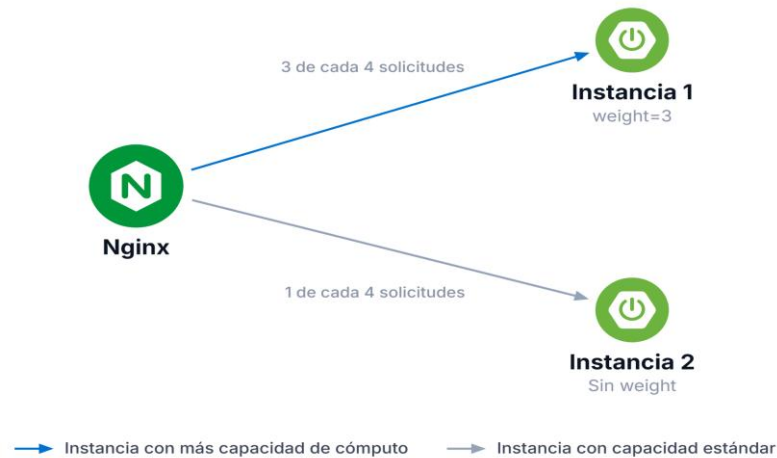


Figura 18. Elaboración propia. Distribución de solicitudes mediante Weighted Round Robin según la capacidad de cada instancia.

Least Connections:

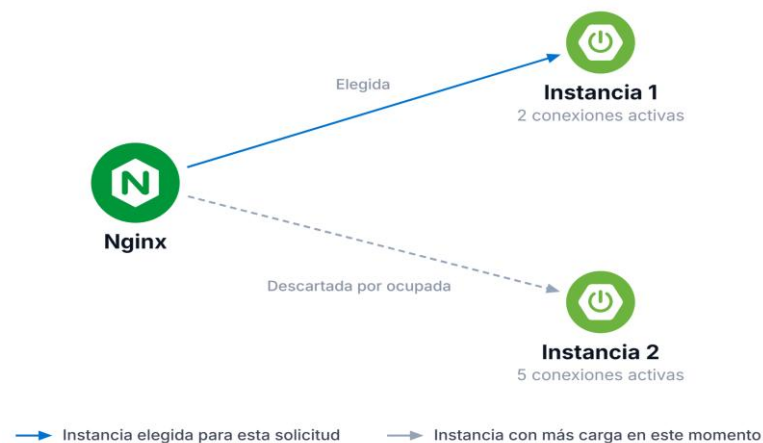


Figura 19. Elaboración propia. Distribución de solicitudes mediante Least Connections según la cantidad de conexiones activas.

IP Hash:

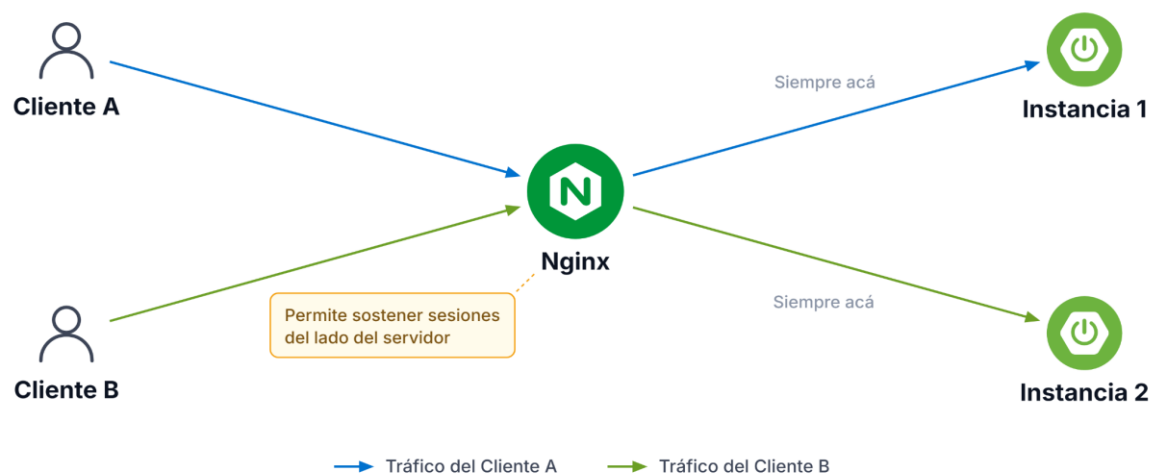


Figura 20. Elaboración propia. Distribución de solicitudes mediante IP Hash para mantener la afinidad de cada cliente con una instancia.

Por ejemplo, si una de las dos instancias de pedidos de la pizzería corriera en un servidor más potente que la otra, alcanzaría con declarar `server pedidos-service-1:8081 weight=3;` para que reciba el triple de solicitudes que la instancia sin peso adicional.

Nginx además detecta instancias que dejan de responder y las retira automáticamente del grupo hasta que vuelven a estar disponibles, sin que haga falta intervención manual.

En producción: la detección básica de Nginx se apoya en fallos de conexión. Para verificaciones más finas (una instancia que responde pero con errores de aplicación) hacen falta health checks activos, disponibles en Nginx Plus o delegados a un orquestador externo.

Implementación práctica: Nginx como servidor web y reverse proxy con Docker

Todo lo visto hasta acá (servidor web, proxy inverso, ruteo por microservicio y balanceo de carga) se puede armar y ejecutar con Docker. Esta sección arma, paso a paso, la infraestructura completa que necesita la pizzería: el frontend Angular ya compilado, y dos microservicios de backend (pedidos y pagos), con un mismo Nginx cumpliendo a la vez los dos roles vistos antes: sirve los archivos estáticos del frontend y reenvía las solicitudes de API al microservicio que corresponda.

Estructura del proyecto

```
reverse-proxy-example/  
├── docker-compose.yml  
├── frontend/  
│   ├── Dockerfile  
│   ├── nginx.conf  
│   └── src/           (proyecto Angular)  
├── pedidos-service/  
└── pagos-service/
```

`frontend/` no es solo el código Angular: incluye su propio Dockerfile y su propio `nginx.conf`, porque el contenedor que sirve el frontend es, al mismo tiempo, el reverse proxy de toda la aplicación. `pedidos-service/` y `pagos-service/` son dos microservicios Spring Boot, cada uno con su propio Dockerfile; `docker-compose.yml` conecta los tres contenedores entre sí.

El contenedor de frontend: build en dos etapas

Antes de que Nginx pueda servir el frontend, el proyecto Angular necesita compilarse (`ng build`), y ese paso requiere Node, una herramienta que no tiene sentido dejar instalada en la imagen final que corre en producción. Por eso el Dockerfile del frontend se escribe en dos etapas: una que compila, y otra que solo copia el resultado.

```
FROM node:20 AS build
WORKDIR /app
COPY . .
RUN npm install && npm run build

FROM nginx:alpine
COPY --from=build /app/dist/pizzeria-frontend /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
```

La primera etapa (**build**) instala las dependencias y genera la carpeta **dist/** con los archivos estáticos ya compilados. La segunda etapa arranca de cero desde una imagen liviana de Nginx, y con **COPY --from=build** copia únicamente esos archivos generados, sin arrastrar Node ni el código fuente de Angular a la imagen final. El resultado es una imagen que solo contiene Nginx y los estáticos, lista para producción.



Figura 21. Elaboración propia. Proceso de construcción y generación de una imagen Docker para una aplicación Angular mediante un build multietapa.

Todo lo que queda a la izquierda de la flecha azul existe solo mientras dura el build y se descarta después: Node, las dependencias y el código fuente nunca llegan a la imagen que corre en producción.

Configuración de Nginx: servidor web y reverse proxy en el mismo archivo

```
server {
    listen 80;
    root /usr/share/nginx/html;
    index index.html;

    location / {
        try_files $uri $uri/ /index.html;
    }
}
```

```
location /api/pedidos/ {
    proxy_pass http://pedidos-service:8081;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
}

location /api/pagos/ {
    proxy_pass http://pagos-service:8082;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
}
}
```

Un mismo bloque `server` cumple los dos roles. El `location /` resuelve el rol de servidor web: sirve los archivos indicados en `root` (los estáticos de Angular). `try_files $uri $uri/ /index.html;` hace que, si la ruta pedida no existe como archivo (por ejemplo `/pedidos/historial`, una ruta que solo existe del lado del router de Angular), Nginx entregue igual `index.html` y sea Angular quien la resuelva en el navegador; sin esta línea, refrescar la página en una ruta interna devolvería un error 404.

Los bloques `location /api/pedidos/` y `location /api/pagos/` resuelven el rol de reverse proxy: interceptan las solicitudes de API antes de que lleguen al bloque `/` y las reenvían (`proxy_pass`) al microservicio correspondiente. Los `proxy_set_header` conservan datos del cliente original (su IP, el host solicitado) que de otro modo se perderían al pasar por el proxy, y que después sirven para logs, trazabilidad o reglas de seguridad del lado del backend.

Orquestación con Docker Compose

```
services:
  frontend:
    build: ./frontend
    ports:
      - "8080:80"
    depends_on:
      - pedidos-service
      - pagos-service

  pedidos-service:
    build: ./pedidos-service
    expose:
      - "8081"

  pagos-service:
    build: ./pagos-service
    expose:
      - "8082"
```

El servicio `frontend` es, en los hechos, el único punto de entrada visible desde afuera: mapea el puerto 8080 del host al puerto 80 del contenedor, y adentro corre la imagen de Nginx armada en el paso anterior. Cada microservicio se expone solo dentro de la red interna que arma Docker Compose, sin publicar sus puertos directamente al host. `depends_on` asegura que el frontend se inicie después de que los microservicios ya estén levantados.

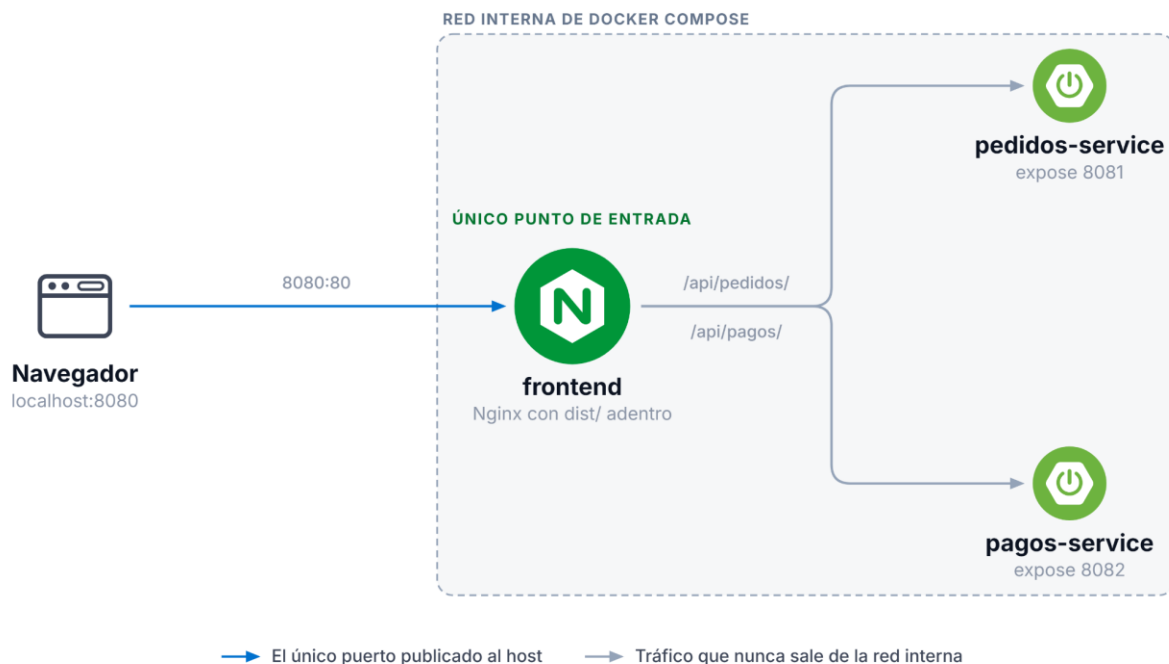


Figura 22. Elaboración propia. Comunicación entre el navegador y microservicios mediante Nginx dentro de una red interna de Docker Compose.

El recuadro punteado marca lo que Docker Compose deja adentro: los microservicios se hablan por nombre dentro de esa red y no tienen ninguna puerta al exterior. La única forma de llegar a ellos desde afuera es atravesando Nginx.

En producción: la detección básica de Nginx se apoya en fallos de conexión. Para verificaciones más finas (una instancia que responde pero con errores de aplicación) hacen falta health checks activos, disponibles en Nginx Plus o delegados a un orquestador externo.

En producción: `depends_on` solo espera a que el contenedor haya iniciado, no a que el microservicio ya esté listo para responder solicitudes. **Para ese caso hace falta un healthcheck con la condición `service_healthy`**, o que el propio Nginx reintente la conexión.

Con esos archivos, `docker-compose up --build` construye las tres imágenes y levanta toda la infraestructura. Accediendo a `http://localhost:8080/` llega el frontend Angular; accediendo a `http://localhost:8080/api/pedidos/` o `http://localhost:8080/api/pagos/`, esa misma Nginx redirige la solicitud al microservicio correspondiente, sin que quien la envía note que hay tres contenedores distintos corriendo puertas adentro.

Configuración dinámica al arrancar el contenedor

Queda un problema pendiente con la imagen que arma el Dockerfile del frontend: las direcciones de `pedidos-service` y `pagos-service` quedaron escritas directamente en el `nginx.conf` que se copió durante el build. Si la pizzería quisiera usar esa misma imagen en un entorno de staging, donde esos servicios tienen otra dirección, tendría que reconstruir la imagen entera solo para cambiar dos líneas de configuración.

La causa es que Angular, a diferencia de un backend que lee variables de entorno en cada arranque, ya generó archivos estáticos en el momento del build: **no queda ningún proceso corriendo del lado del frontend que pueda consultar el entorno en tiempo real. Todo lo que se compiló queda fijo dentro de la imagen.**

Una forma habitual de resolver esto es no copiar el `nginx.conf` final durante el build, sino una plantilla con variables, y completarla recién cuando el contenedor arranca, con la utilidad `envsubst`.

```
location /api/pedidos/ {
    proxy_pass ${PEDIDOS_URL};
}

envsubst < /etc/nginx/nginx.conf.template > /etc/nginx/conf.d/default.conf
nginx -g 'daemon off;'
```

El primer archivo es la plantilla que viaja dentro de la imagen, con `${PEDIDOS_URL}` todavía sin resolver. El segundo es el script de arranque del contenedor: `envsubst` reemplaza esa variable por el valor real que llega desde el entorno de ejecución, y recién después arranca Nginx. Con este cambio, la misma imagen Docker sirve para staging y para producción; lo único que cambia es la variable de entorno con la que se levanta el contenedor, sin volver a compilar nada.

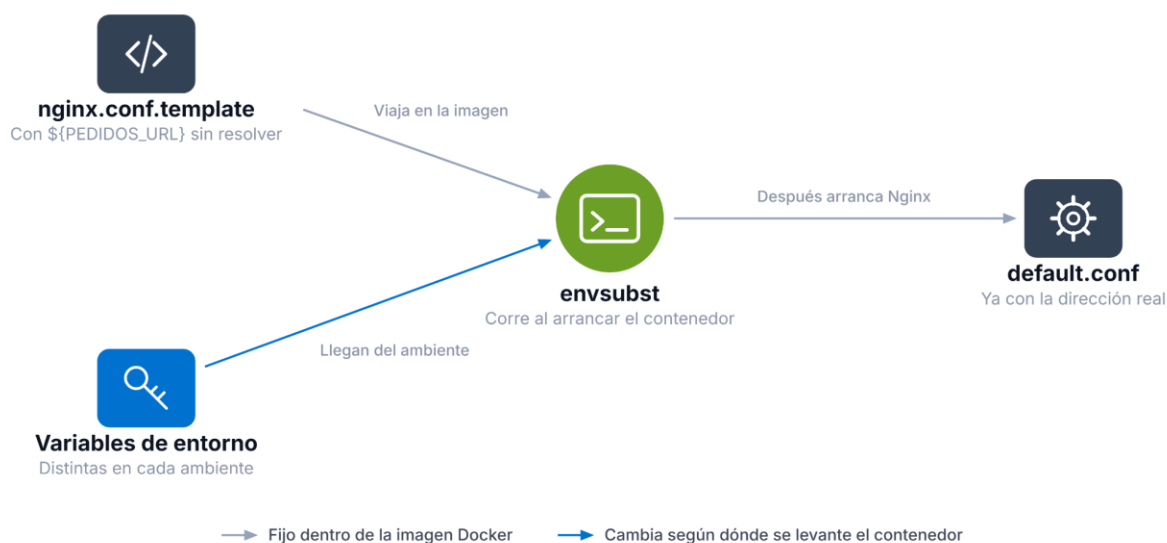


Figura 23. Elaboración propia. Generación dinámica de la configuración de Nginx mediante variables de entorno y `envsubst`.

La plantilla es siempre la misma, viaje donde viaje la imagen; lo único que cambia entre staging y producción es lo que entra por la flecha azul.

Con esta infraestructura armada, la pizzería tiene resuelto cómo se sirve y organiza su sistema completo, frontend y backend incluidos. Queda una pregunta distinta: cuando hay una versión nueva del backend o del frontend lista para publicarse, ¿cómo se pasa de la versión actual a la nueva sin interrumpir el servicio? Esa pregunta ya no tiene que ver con la infraestructura que atiende el tráfico, sino con el proceso de publicar cambios, y es el tema del resto de la unidad.

Qué es el despliegue y por qué es una etapa crítica

Publicar una versión nueva de un sistema no es simplemente copiar archivos nuevos al servidor. El despliegue es el proceso controlado mediante el cual una aplicación o un servicio se instala, se configura y se pone en funcionamiento en un entorno determinado: prepara la infraestructura, configura variables y datos sensibles, ejecuta los scripts de inicialización que hagan falta, y expone el sistema de forma segura a sus usuarios.

Un mal despliegue tiene consecuencias concretas. Un tiempo de inactividad, un error visible o una funcionalidad rota afectan directamente la percepción de quien usa el sistema; en un comercio electrónico o un servicio financiero, unos minutos de caída pueden traducirse en pérdidas económicas reales. Por eso las organizaciones que logran desplegar rápido y con confianza responden mejor a las demandas del mercado, y por eso el despliegue automatizado y frecuente es uno de los pilares de metodologías como Continuous Delivery y DevOps.

Netflix es un caso conocido: realiza cientos de despliegues por día gracias a una infraestructura automatizada. Cada versión nueva pasa primero por un lanzamiento controlado (una fracción chica del tráfico se dirige al servicio nuevo) y, si las métricas de error y rendimiento se mantienen dentro de los umbrales esperados, el despliegue avanza hasta cubrir a todos los usuarios; si algo falla, el sistema revierte automáticamente. Ese patrón tiene nombre propio (Canary) y se desarrolla más adelante en esta unidad.

Un despliegue profesional, además de instalar el artefacto (una imagen Docker, un archivo JAR, una función serverless), contempla varios aspectos: el versionado y la trazabilidad de cada release (saber qué versión corre en cada entorno y cuándo se liberó), la estrategia de liberación (de qué manera se pasa de la versión anterior a la nueva), la observabilidad posterior al release (métricas, logs y alertas que detecten anomalías rápido) y un plan de rollback (cómo revertir si algo sale mal).

El despliegue en el ciclo de vida del software

El despliegue solía ubicarse al final del ciclo de desarrollo, como una etapa aislada que ocurría una sola vez, cuando todo el trabajo de programación y pruebas ya estaba terminado. Con la llegada de los enfoques ágiles, DevOps y Continuous Delivery, pasó a ser una actividad recurrente e integrada dentro de un ciclo que se repite en cada iteración.

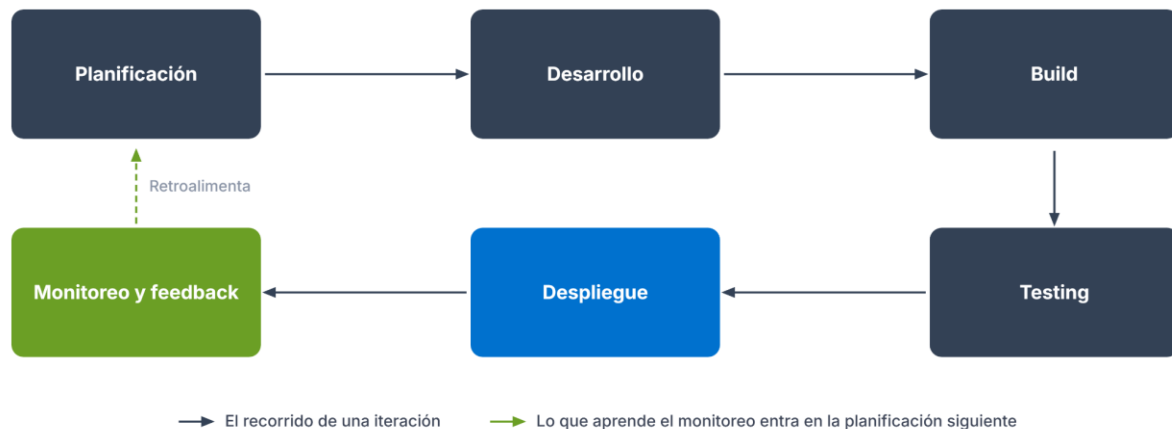


Figura 24. Elaboración propia. Ciclo iterativo de desarrollo, despliegue, monitoreo y retroalimentación continua.

Este ciclo no es lineal sino iterativo: cada vuelta genera una versión nueva lista para desplegar, y el monitoreo posterior retroalimenta directamente a la planificación siguiente. Empresas como Amazon o Google llegan a hacer miles de despliegues diarios gracias a arquitecturas de microservicios (como la que armó la pizzería en el bloque anterior) y pipelines automatizados que permiten liberar cambios chicos con un riesgo mínimo. La clave está en acortar ese ciclo, para que las mejoras lleguen a quien usa el sistema casi de inmediato.

Estrategias de despliegue: Blue-Green y Canary

Publicar una versión nueva no es una actividad uniforme: existen varias estrategias, también llamadas patrones de release, que equilibran de forma distinta la estabilidad del sistema con la velocidad de entrega. La elección depende del tipo de aplicación, del entorno y de cuánto riesgo está dispuesto a asumir el equipo. Las dos que siguen son las más extendidas en la industria.

Blue-Green Deployment

Imaginá que la pizzería tiene lista una nueva versión de su backend de pedidos, pero reemplazarla de golpe significa arriesgarse a que un error tumbé el sistema justo un sábado a la noche. El enfoque Blue-Green evita ese riesgo manteniendo dos entornos idénticos: uno (Blue) es la versión activa y estable que atiende a los usuarios; el otro (Green) contiene la versión nueva, ya lista pero todavía sin tráfico real.

La versión nueva se prepara y se valida en el entorno Green mientras Blue sigue funcionando con normalidad. Una vez verificada, el tráfico se redirige de Blue a Green (en la

infraestructura de la pizzería, esto es exactamente lo que el reverse proxy de Nginx puede resolver, cambiando a qué entorno apunta). Si algo falla, alcanza con redirigir el tráfico de vuelta a Blue para revertir el cambio de forma casi instantánea.

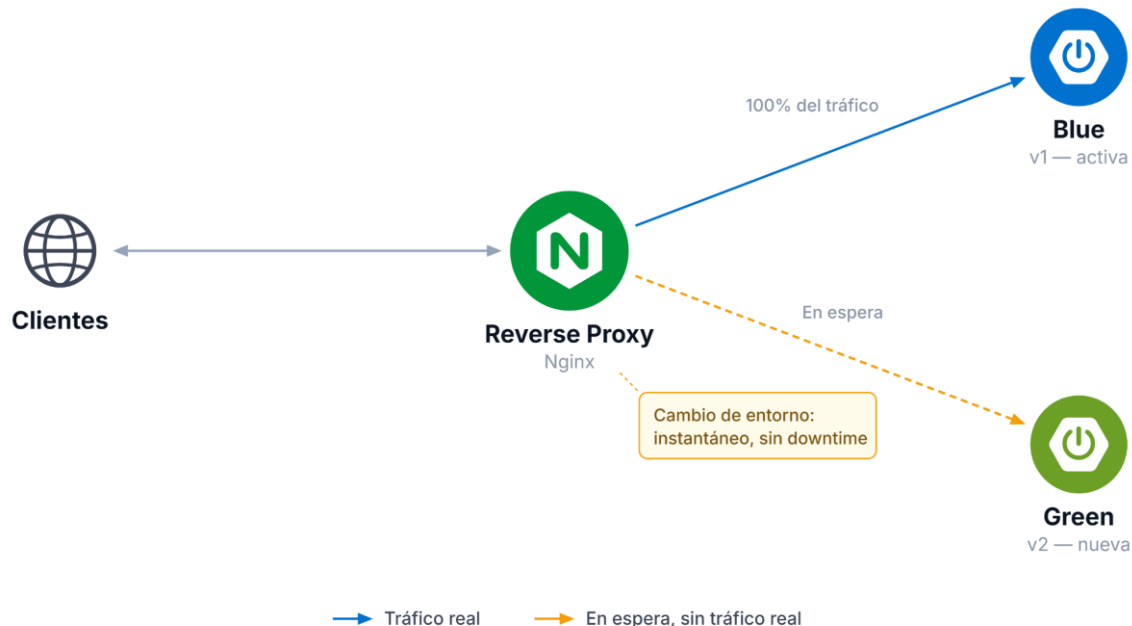


Figura 25. Elaboración propia. Estrategia de despliegue Blue-Green con conmutación de tráfico mediante Nginx.

La principal ventaja de Blue-Green es que garantiza cero tiempo de inactividad y un rollback casi inmediato, algo valioso para sistemas donde la disponibilidad es prioritaria. El costo es mantener dos entornos completos activos al mismo tiempo, lo que duplica la infraestructura necesaria; además, se vuelve más difícil de aplicar cuando la versión nueva requiere migraciones de base de datos, porque hay que mantener sincronizados dos esquemas de datos mientras dura la transición.

Canary Deployment

El despliegue canario toma su nombre de las canarias que los mineros llevaban para detectar gases tóxicos antes de que se volvieran peligrosos para las personas. La idea es parecida: en vez de exponer a todos los usuarios a la versión nueva de una sola vez, se libera primero a un grupo pequeño (por ejemplo, un uno por ciento del tráfico), mientras el resto sigue en la versión estable.

Durante ese tiempo se monitorean métricas clave (errores, tiempos de respuesta, consumo de recursos) para evaluar si la versión nueva se comporta bien. Si los resultados son favorables, el porcentaje de tráfico dirigido a la versión nueva aumenta de forma progresiva hasta cubrir a todos los usuarios; si aparece un problema, se revierte y el tráfico vuelve por completo a la versión anterior.

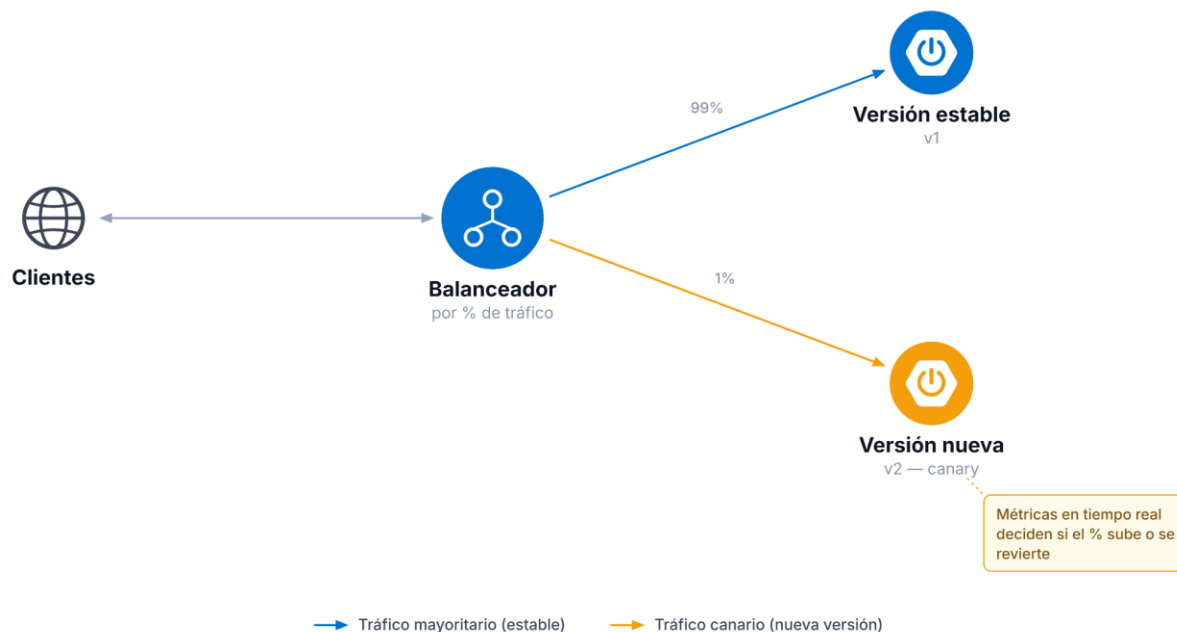


Figura 26. Elaboración propia. Estrategia de despliegue Canary mediante distribución porcentual del tráfico entre una versión estable y una nueva versión.

Canary detecta errores en producción con un impacto mínimo, porque solo una fracción chica de usuarios queda expuesta si algo sale mal, y permite evaluar el comportamiento real del sistema bajo carga real, no simulada. A cambio, necesita una infraestructura más sofisticada que Blue-Green (balanceadores capaces de repartir tráfico por porcentaje y monitoreo en tiempo real), y su configuración inicial suele ser más compleja.

Estrategias de despliegue: Rolling Update, A/B Testing, Shadow Deployment y Feature Flags

Blue-Green y Canary son las estrategias más conocidas, pero no las únicas. El resto resuelve variantes del mismo problema (cómo pasar de una versión a otra sin arriesgar el sistema) con matices distintos, y en la práctica profesional conviene reconocerlas para elegir la que mejor se ajusta a cada situación.

Rolling Update

En lugar de mantener dos entornos completos como en Blue-Green, el Rolling Update reemplaza las instancias de la versión vieja por instancias de la versión nueva de a poco, una o unas pocas por vez, hasta que todas quedaron actualizadas. Durante la transición conviven instancias de ambas versiones atendiendo tráfico al mismo tiempo.

El costo principal es que, durante la ventana de actualización, algunas solicitudes son atendidas por la versión vieja y otras por la nueva, lo que exige que ambas puedan convivir sin romper la compatibilidad (si cambia el formato de una respuesta de la API, las dos versiones tienen que poder convivir sin romper al frontend). A cambio, no necesita duplicar toda la infraestructura como Blue-Green: solo reemplaza gradualmente lo que ya existe.



Figura 27. Elaboración propia. Despliegue Rolling Update.

Podemos identificar los siguientes pasos:

Paso 1: Ambas instancias con v1

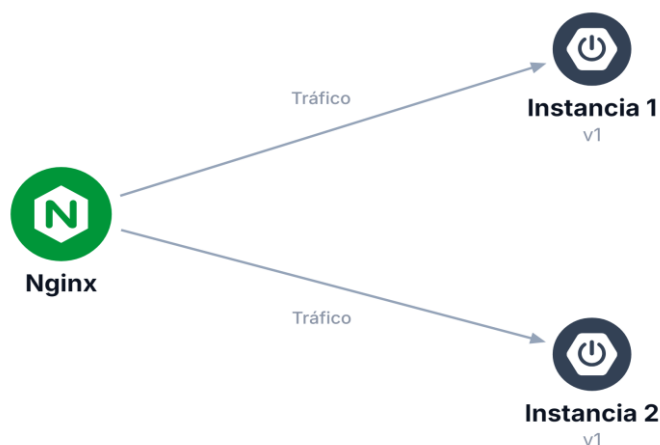


Figura 28. Elaboración propia. Distribución de tráfico mediante Nginx entre múltiples instancias.

Paso 2: Una de las instancias con v2

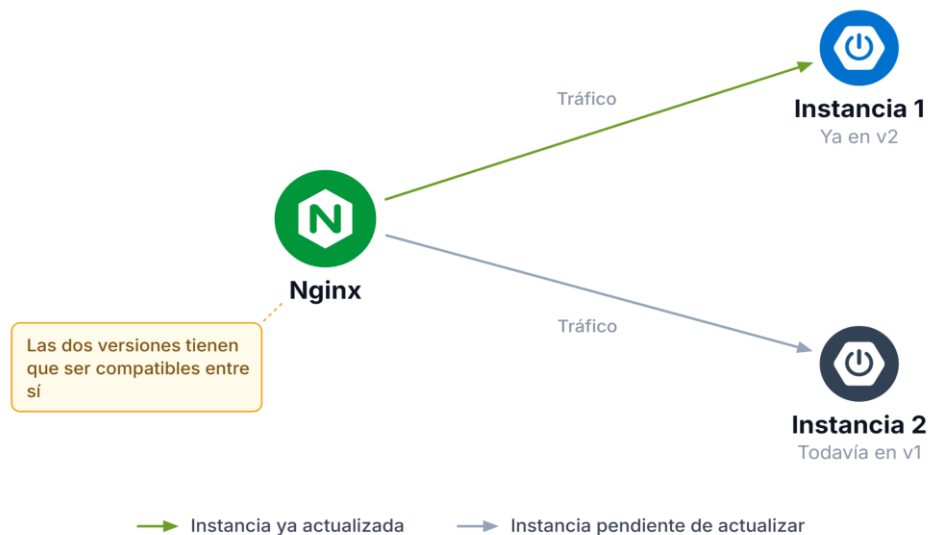


Figura 29. Elaboración propia. Convivencia temporal de versiones durante un Rolling Update.

Paso 3: Ambas instancias con v2

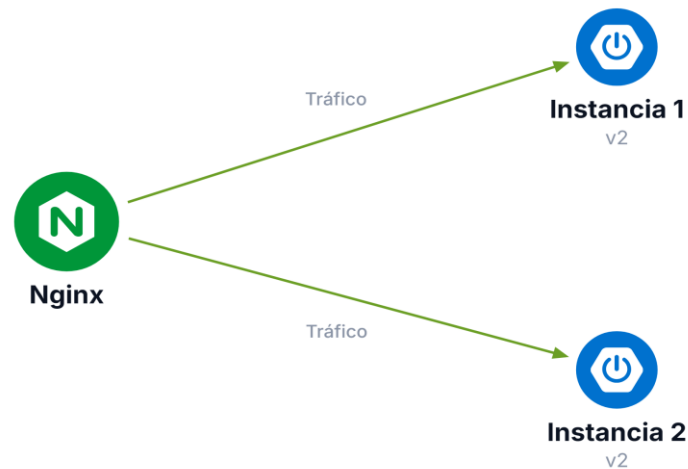


Figura 31. Elaboración propia. Distribución de tráfico entre instancias actualizadas a la versión v2.

A/B Testing Deployment

A primera vista, A/B Testing se parece a Canary: también hay dos versiones activas al mismo tiempo, recibiendo tráfico real. La diferencia está en el objetivo. Canary mide estabilidad técnica (errores, tiempos de respuesta) para decidir si una versión es segura; A/B Testing mide comportamiento de negocio o de experiencia de usuario (por ejemplo, qué versión de la pantalla de pedido logra más conversiones) para decidir cuál conviene, no cuál es más estable. Por eso A/B Testing suele mantenerse activo más tiempo, incluso semanas, mientras que un Canary típicamente se resuelve en horas o días una vez que la versión nueva demostró ser estable.

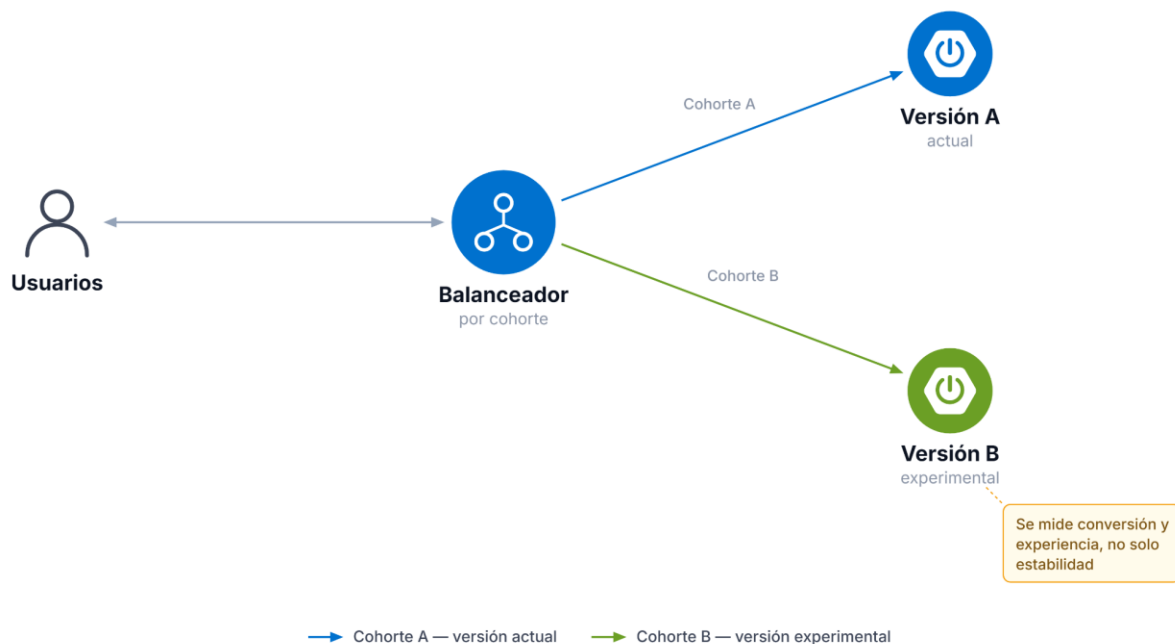


Figura 32. Elaboración propia. Distribución de usuarios por cohortes para realizar pruebas A/B entre distintas versiones.

Shadow Deployment

Shadow Deployment lleva la cautela un paso más allá: el tráfico real se duplica y se envía también a la versión nueva, pero la respuesta de esa versión nunca llega a quien la generó, se descarta. Solo se usa para comparar cómo se hubiera comportado la versión nueva frente al tráfico real, sin ningún riesgo de que un error llegue a afectar a alguien.

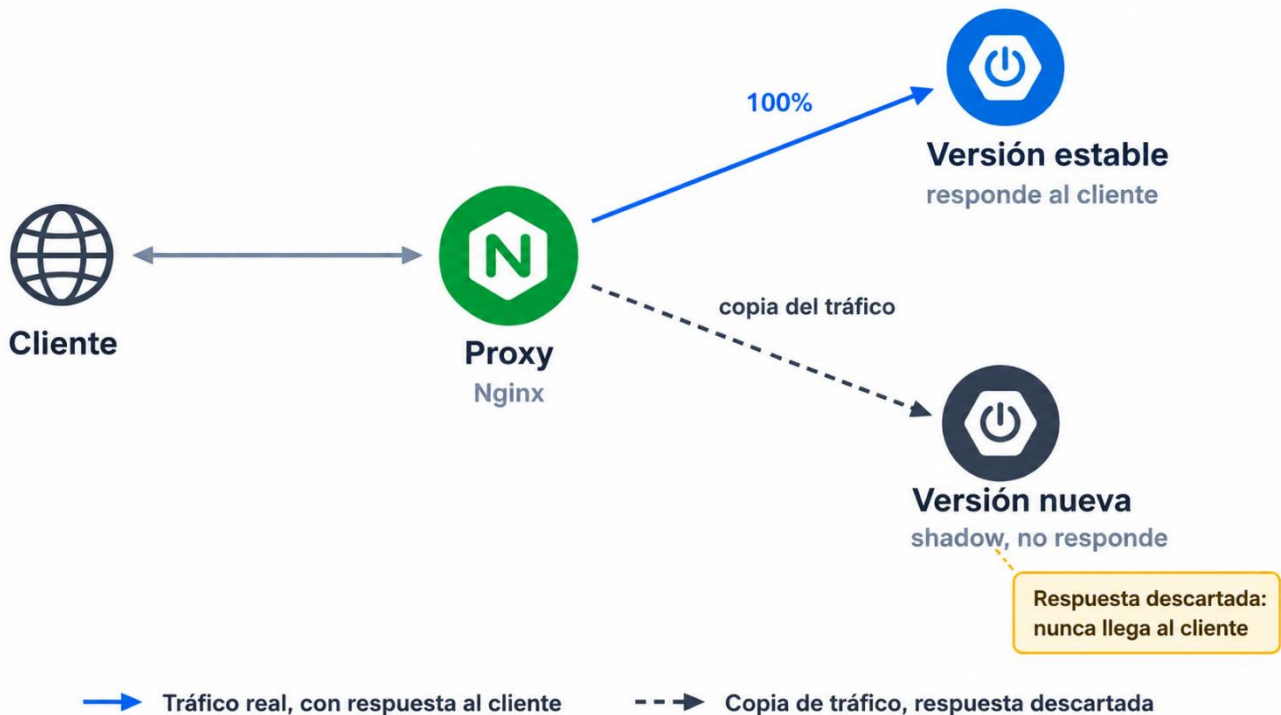


Figura 33. Elaboración propia. Estrategia Shadow Deployment mediante duplicación del tráfico hacia una nueva versión sin afectar al cliente.

La ventaja es que permite validar rendimiento y comportamiento con datos completamente reales y sin ningún riesgo para el usuario. El costo es la complejidad de infraestructura: hace falta duplicar el tráfico sin duplicar sus efectos, algo especialmente delicado cuando la operación no es de solo lectura (duplicar un pedido o un pago real sería un problema serio, no una simulación).

Feature Flags

Las estrategias anteriores deciden cuándo un servidor corre una versión u otra. Feature Flags resuelve un problema distinto: cómo activar o desactivar una funcionalidad puntual dentro de una misma versión ya desplegada, sin necesidad de un nuevo despliegue. El código de la funcionalidad nueva ya está en producción, pero queda apagado detrás de una bandera (flag) hasta que se decide activarlo, para todos los usuarios o solo para un grupo.

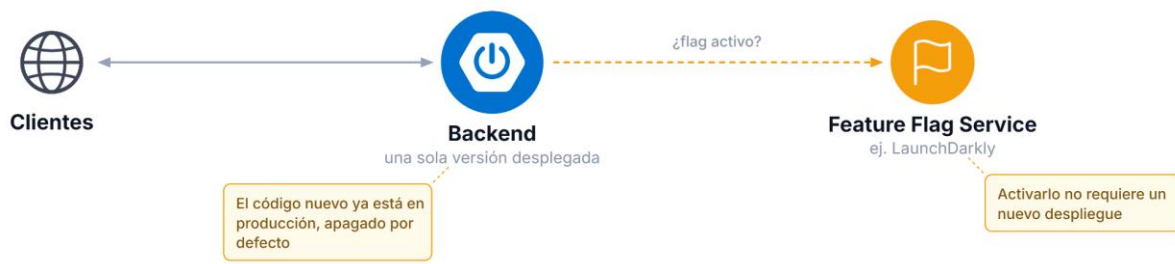


Figura 34. Elaboración propia. Uso de Feature Flags para habilitar o deshabilitar funcionalidades sin necesidad de un nuevo despliegue.

Esto separa dos decisiones que en las estrategias anteriores estaban atadas: cuándo se despliega el código (una decisión técnica) y cuándo se activa la funcionalidad (una decisión de negocio). La pizzería podría, por ejemplo, desplegar el código de un nuevo medio de pago ya integrado pero apagado, y activarlo recién cuando el equipo comercial lo decida, sin depender de otro despliegue.

Herramientas para el despliegue

Todo lo anterior (estrategias, infraestructura, pipelines) se apoya en un ecosistema de herramientas que puede agruparse en cuatro categorías.

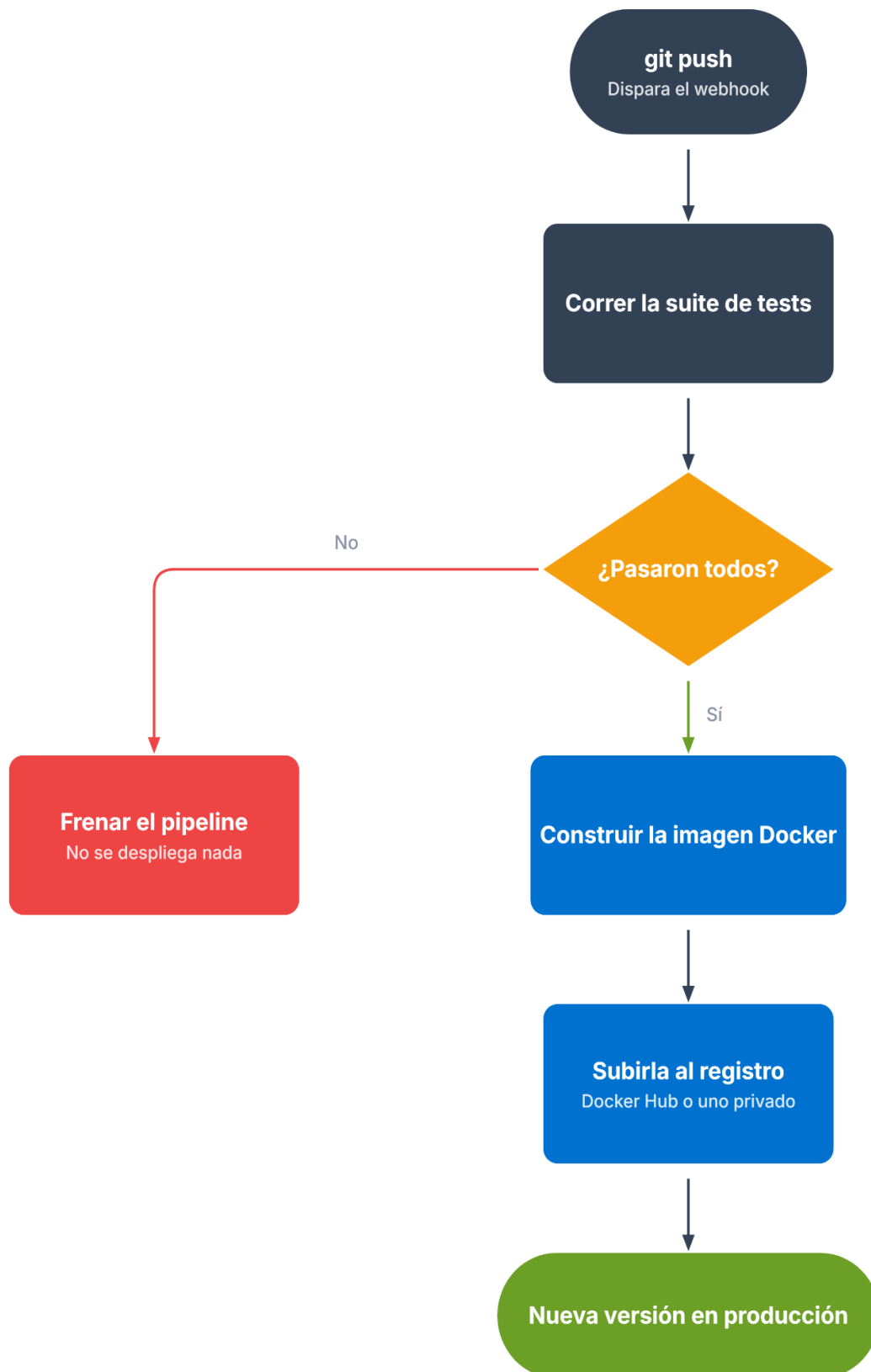
Infraestructura e inmutabilidad

Antes de desplegar cualquier estrategia hace falta infraestructura que crear y reproducir de forma confiable. Herramientas de infraestructura como código (Terraform, Ansible) permiten describir servidores, redes y configuraciones en archivos versionables, en vez de armarlos a mano cada vez. Sobre esa base se apoya la idea de infraestructura inmutable: en lugar de modificar un servidor existente, se construye una imagen nueva (por ejemplo, una imagen Docker, como la que arma Nginx con el build de Angular adentro) y se reemplaza el servidor entero. Esto evita el problema clásico de la configuración que funciona en un servidor y no en otro, por diferencias acumuladas con el tiempo.

Pipelines de integración y despliegue continuo

Un pipeline de CI/CD automatiza el recorrido del ciclo de vida visto antes (build, testing, despliegue) cada vez que se sube un cambio de código. Herramientas como GitHub Actions, GitLab CI o Jenkins ejecutan esos pasos de forma automática: compilan la aplicación, corren la suite de tests y, si todo pasa, disparan el despliegue según la estrategia elegida (Blue-Green, Canary o la que corresponda). Sin este tipo de automatización, aplicar cualquiera de las estrategias anteriores a mano, en cada release, sería poco práctico.

Un caso concreto ayuda a ver ese recorrido en acción. Cuando alguien hace `git push` a un repositorio, ese evento dispara automáticamente, vía webhook, una ejecución del pipeline configurado:



→ El cambio avanza hacia producción → El pipeline corta y nada llega al servidor

Figura 35. Elaboración propia. Flujo de integración y despliegue continuo desde el git push hasta la nueva versión en producción.

El pipeline clona el código, corre la suite de tests y, si pasa, construye una imagen Docker nueva (por ejemplo, la que arma Nginx con el frontend adentro, vista en la sección de implementación práctica) y la sube a un registro de imágenes (Docker Hub, o un registro privado de la nube). Desde ahí, el servidor de producción, o un orquestador, descarga esa imagen nueva y reemplaza el contenedor que estaba corriendo, sin intervención manual en ningún paso intermedio. Es este mismo pipeline el que, en un despliegue Blue-Green o Canary, dispara la estrategia elegida una vez que la imagen nueva está lista.

Secretos y configuración

Ninguna de las estrategias anteriores funciona bien si las credenciales (contraseñas de base de datos, tokens de API) quedan hardcodeadas en el código o en archivos de configuración versionados junto con el proyecto. Herramientas como Vault o los gestores de secretos de cada proveedor de nube permiten inyectar esos valores en tiempo de ejecución, separados del código fuente, y rotarlos sin necesidad de un nuevo despliegue.

Monitoreo y rollback automático

El último eslabón es el que cierra el círculo con las estrategias de despliegue: sin monitoreo, ninguna decisión de avanzar o revertir un Canary o un Blue-Green puede tomarse a tiempo. Herramientas como Prometheus (para métricas) y Grafana (para visualizarlas), junto con sistemas de alertas, permiten detectar cuando una versión nueva se comporta peor que la anterior y, en las infraestructuras más maduras, disparar un rollback automático sin esperar a que una persona lo note.

Categoría	Qué resuelve	Herramientas de referencia
Infraestructura e inmutabilidad	Crear y reproducir infraestructura de forma confiable	Terraform, Ansible, Docker
Pipelines de CI/CD	Automatizar build, testing y despliegue en cada cambio	GitHub Actions, GitLab CI, Jenkins
Secretos y configuración	Mantener credenciales fuera del código fuente	Vault, gestores de secretos de la nube
Monitoreo y rollback automático	Detectar anomalías y decidir si una versión sigue o se revierte	Prometheus, Grafana

Tabla 2. Elaboración propia. Categorías y herramientas de referencia para automatización, despliegue y monitoreo de infraestructura.

En resumen

La infraestructura que arma Nginx (un punto de entrada único, capaz de servir contenido estático, reenviar a servidores de aplicación, rutear entre microservicios y balancear carga) es, en la práctica, el lugar donde las estrategias de despliegue se ejecutan: un Blue-Green cambia a qué entorno apunta ese reverse proxy, un Canary ajusta qué porcentaje de tráfico llega a cada versión a través del balanceador, y un Rolling Update reemplaza, una por una, las instancias que el gateway ya conoce. Entender la arquitectura de la primera mitad de esta unidad es lo que permite razonar con criterio sobre las estrategias de la segunda.

Estrategia	Riesgo	Complejidad de infraestructura	Se usa cuando
Blue-Green	Bajo (rollback inmediato)	Alta (dos entornos completos)	La disponibilidad es prioritaria y se puede duplicar infraestructura
Canary	Bajo (exposición gradual)	Alta (balanceo por porcentaje, monitoreo)	Se quiere validar estabilidad técnica con impacto mínimo
Rolling Update	Medio (conviven versiones)	Media (reemplazo gradual, sin duplicar todo)	No se puede duplicar infraestructura pero se acepta la convivencia de versiones
A/B Testing	Bajo (similar a Canary)	Alta	Se quiere medir negocio o experiencia de usuario, no solo estabilidad
Shadow Deployment	Mínimo (la respuesta se descarta)	Alta (duplicar tráfico sin duplicar efectos)	Se quiere validar con tráfico real sin ningún riesgo para el usuario
Feature Flags	Bajo (activación controlada)	Baja o media (requiere gestión de flags)	Se quiere separar el despliegue de código de la activación de una funcionalidad

Atribución-No Comercial-Sin Derivadas



Se permite descargar esta obra y compartirla, siempre y cuando no sea modificado y/o alterado su contenido, ni se comercialice. Referenciarlo de la siguiente manera:

Universidad Tecnológica Nacional Facultad Regional Córdoba (S/D). Material para la Tecnicatura Universitaria en Programación, modalidad virtual, Córdoba, Argentina.